MASTER TECHNICAL REQUIREMENTS DOCUMENT

Hospital Management System (HMS) ERP

Detailed Production-Level Technical Specification

Project	Hospital Management System (HMS) ERP
Architecture	Decoupled Monolithic Backend + Vue.js SPA + Native Android Client
Backend	Laravel REST API
Frontend	Vue.js SPA
Mobile	Native Android (Kotlin/Java)
Database	MySQL / InnoDB
Authentication	Laravel Sanctum
Notifications	Firebase Cloud Messaging / SMS Gateway
Target Delivery	25 Working Days
Basis	User-provided Master TRD

Document basis: This TRD expands the supplied Master TRD into implementation-level requirements while preserving its architecture, terminology, modules, API direction and stated constraints.

1. System Architecture & Technology Stack

The HMS uses a decoupled monolithic backend: Laravel is the central REST API and business-logic layer, Vue.js is the web SPA, and the native Android Doctor Application consumes the same API. MySQL/InnoDB is the transactional persistence layer. Firebase FCM, SMS Gateway, Redis/queue workers, local/cloud storage and scheduled jobs support asynchronous operations.

1.1 Architecture Principles

- Laravel is the single authoritative business-logic layer.
- Web and Android clients communicate through versioned HTTPS REST APIs.
- Laravel Sanctum provides stateless token authentication.
- Server-side RBAC is authoritative; frontend visibility is not a security boundary.
- Critical multi-record operations use database transactions.
- SMS, FCM, PDF and other long-running external operations should use queues where appropriate.
- Clients never connect directly to MySQL.

1.2 Technology Stack

Layer	Technology	Requirement
Backend	Laravel (Latest)	Headless REST API, validation, authorization, business logic, jobs and notifications
Web	Vue.js	SPA using Vue Router, Pinia and Axios
Mobile	Native Android	Kotlin/Java, REST API and Firebase FCM
Database	MySQL 8.0 / InnoDB	Transactions, indexed foreign keys and relational integrity
Authentication	Laravel Sanctum	Bearer-token API authentication
Server	Nginx + PHP 8.x-fpm	Reverse proxy, HTTPS and Laravel runtime
Queue	Redis / Laravel Queue	FCM, SMS and asynchronous jobs
PDF	Dompdf / Browsershot	Laboratory report generation
OS	Ubuntu LTS	Cloud server platform
Source	GitHub	Version-controlled source and deployment workflow

2. Core Application Components

The backend, web application and mobile application are separate client/runtime concerns but share one versioned API contract.

2.1 Laravel Backend

- Authentication and token lifecycle.
- RBAC and permission enforcement.
- Patient, appointment, OPD/IPD and bed workflows.
- Billing, ledger and financial processing.
- Pharmacy and laboratory workflows.
- Referral commission and wallet processing.
- File/PDF handling.
- Queues, notifications, scheduler and audit logging.

2.2 Vue.js SPA

- Role-specific dashboards and navigation.
- Vue Router route protection.
- Pinia authentication/permission and operational state.
- Axios API service layer.
- Reusable forms, tables, dialogs, pagination and feedback states.
- Responsive desktop/tablet interfaces.

2.3 Android Doctor App

- Secure login.
- Today's appointments.
- Patient history and medical reports.
- Digital prescriptions.
- Admission and follow-up information.
- Earnings summary.
- Profile management.
- Firebase FCM notifications.

3. Database Architecture

The supplied core schema is retained as the logical foundation. Supporting tables may be introduced to implement the stated requirements safely without changing the business scope.

3.1 Core Entities

Entity	Purpose	Relationships
users	Credentials, role, status and profile	role_id; doctor/franchise profile
patients	UHID and demographics	appointments, admissions, prescriptions, lab requests, invoices
doctors	Doctor profile	user, department, appointments, prescriptions, referrals
appointments	Appointment/token lifecycle	patient, doctor
ipd_admissions	IPD lifecycle	patient, bed
beds	Physical bed state	IPD admissions
prescriptions	Prescription header	patient, doctor, appointment
prescription_items	Prescription lines	prescription, medicine
medicines	Medicine inventory	category and inventory transactions
lab_tests	Test master	category
lab_requests	Patient test workflow	patient, doctor, test
invoices	Billing header	patient and module
accounts_ledger	Income/expense entries	billing and financial operations
franchises	Partner/wallet	user, referrals
referrals	Referral/commission record	franchise, doctor, patient

3.2 Data Integrity Rules

- UHID is unique and indexed.
- Foreign keys are indexed and constrained.
- Money uses fixed-precision decimal values.
- Critical multi-row updates are transactional.
- Financial records are not silently hard-deleted after posting.
- Frequently queried fields such as UHID, phone, barcode, appointment date, doctor ID, patient ID and status are indexed.
- Status transitions are validated by the server.

3.3 Production Supporting Tables

- roles, permissions and role/permission mapping tables.
- departments and master-data tables.
- invoice_items and payment/refund records.
- inventory transaction records.
- bed transfer/history records.
- notification delivery records where required.

- audit_logs containing user, module, IP, timestamp and action payload.

4. Authentication, RBAC & Security

4.1 Authentication Flow

- POST /api/v1/auth/login validates credentials and account status.
- Laravel Sanctum issues a Bearer token.
- The response returns user profile and explicit permission scope.
- Protected clients send the token over HTTPS.
- POST /api/v1/auth/logout revokes the current API token.

4.2 Security Requirements

- HTTPS/SSL across all interfaces.
- Server-side authorization on every protected operation.
- Validation on every write endpoint.
- Rate limiting for authentication and abuse-sensitive endpoints.
- No production stack traces or secrets in API responses.
- Sensitive database fields encrypted where required.
- Uploaded files validated for type and size.
- Production secrets externalized from source control.

5. REST API Standards

All endpoints use the /api/v1 namespace. Breaking changes should use a new API version.

5.1 API Response Contract

Response	Expected Behavior
Success	data, message and meta where pagination/metadata is applicable
Validation	HTTP 422 with message and field-level errors
Unauthorized	HTTP 401 with safe message
Forbidden	HTTP 403 with safe message
Not Found	HTTP 404 with safe message
Conflict	Appropriate conflict response for state/concurrency conflicts
Server Error	Safe generic response; detailed error remains server-side

5.2 Required API Inventory

Area	Endpoint	Purpose
Auth	POST /api/v1/auth/login	Login + permission scope
Auth	POST /api/v1/auth/logout	Revoke token
Patients	POST /api/v1/patients	Create patient + UHID
Patients	GET /api/v1/patients/{id}/history	Aggregated history
Beds	GET /api/v1/beds/status	Occupancy matrix
Beds	POST /api/v1/beds/transfer	Atomic transfer
Appointments	POST /api/v1/appointments	Appointment + daily token
Billing	POST /api/v1/invoices/generate	GST-aware consolidated invoice
Pharmacy	GET /api/v1/pharmacy/inventory	Paginated inventory
Lab	POST /api/v1/lab/reports/upload	Generate/store report PDF
Referral	POST /api/v1/referrals/process	Commission calculation
Franchise	POST /api/v1/franchise/withdraw	Withdrawal request

Area	Endpoint	Purpose
Doctor	GET /api/v1/doctor/appointments/today	Doctor schedule
Doctor	POST /api/v1/doctor/prescriptions	Digital prescription
Doctor	GET /api/v1/doctor/earnings	Date-range earnings

6. Patient & Bed Management

6.1 Patient Registration

- POST /api/v1/patients creates a patient.
- UHID format: UHID-YYYYMMDD-XXXX.
- UHID must be unique and indexed.
- Demographic data is validated before persistence.
- Creation is auditable.

6.2 Patient History

GET /api/v1/patients/{id}/history returns an aggregated medical record containing prescriptions, laboratory tests and IPD admissions, subject to authorization.

6.3 Bed Status

GET /api/v1/beds/status returns occupancy grouped by ICU, General Ward, Private and Deluxe. Bed status values are Available, Occupied and Maintenance.

6.4 Atomic Bed Transfer

- Validate old and new bed.
- Verify destination is Available.
- Begin DB transaction.
- Set old bed Available.
- Set new bed Occupied.
- Update related admission.
- Write transfer/audit record.
- Commit only when all operations succeed; otherwise rollback.

7. Appointment, OPD & IPD

7.1 Appointment Workflow

- Validate patient, doctor, date and appointment type.
- Validate scheduling/business state.
- Generate daily sequential token safely under concurrency.
- Persist appointment.
- Dispatch asynchronous SMS reminder.
- Return appointment and token.

7.2 Token Concurrency

Daily sequential tokens must be protected against concurrent creation. The implementation should use a transactional/locking strategy rather than an unsafe application-only increment.

7.3 IPD Admission

- Create admission.
- Validate bed availability.
- Assign bed and mark it Occupied transactionally.
- Maintain admission/discharge dates and status.
- Use the atomic transfer workflow for subsequent transfers.

8. Billing & Financial Accounting

8.1 Invoice Generation

- POST /api/v1/invoices/generate accepts billable items.
- Server calculates subtotal, discount, GST and final amount.
- Invoice number is unique.
- Invoice items should be stored separately for traceability.
- Payment state remains consistent with payment records.
- accounts_ledger entry is coordinated transactionally with billing.

8.2 Financial Controls

- Server-side calculation is authoritative.
- Decimal arithmetic is used for money.
- Posted financial records are auditable.
- Invoice and ledger writes are coordinated in a DB transaction.
- Refund/adjustment workflows must preserve auditability.

9. Pharmacy & Laboratory

9.1 Pharmacy

- Inventory endpoint supports pagination.
- Barcode search is supported.
- Low-stock threshold flags are returned.
- Expiry warnings are returned.
- Inventory quantity changes are traceable.
- Medicine billing connects to invoice/payment workflows.

9.2 Laboratory

- Test categories and tests are maintained as master data.
- Lab request links patient, doctor and test.
- Sample status is tracked.
- Dompdf/Browsershot generates PDF.
- report_pdf_path stores the generated report path.
- Successful report generation changes request status to Ready.

10. Franchise / Channel Partner Commission

10.1 Commission Processing

- POST /api/v1/referrals/process evaluates bill details.
- Rules support Doctor-wise, Service-wise, Fixed and Surgery-wise models.
- Commission is calculated server-side.
- Referral status and wallet credit are transactional.
- Duplicate processing must be prevented.
- Commission calculation is auditable.

10.2 Wallet Withdrawal

- POST /api/v1/franchise/withdraw creates a request.
- Requested amount cannot exceed eligible balance.
- Withdrawal state is tracked.
- Administrative settlement is auditable.

11. Doctor Android Application

Feature	Technical Requirement
Login	Sanctum-authenticated REST API
Appointments	GET /api/v1/doctor/appointments/today

Feature	Technical Requirement
Patient History	Authorized patient-history retrieval
Prescription	POST /api/v1/doctor/prescriptions
Reports	Authorized report access
Admissions	Relevant IPD details
Follow-up	Follow-up schedule
Notifications	Firebase FCM
Earnings	GET /api/v1/doctor/earnings
Profile	Permitted profile management

11.1 Mobile Security

- Secure token storage.
- Doctor-scoped server-side authorization.
- No direct database connection.
- FCM payloads should minimize sensitive clinical data.
- Logout/revocation support.

12. Notifications, Queues & Scheduler

12.1 Asynchronous Work

- SMS appointment reminders.
- FCM push notifications.
- PDF/report generation when expensive.
- Other external service operations that should not block critical API requests.

12.2 Reliability

- Retry transient failures.
- Log failed jobs.
- Make external notifications as idempotent as practical.
- Run workers under a production process supervisor.
- Monitor queue backlog and failed jobs.

13. File Storage & PDF Generation

- Patient and laboratory documents use controlled storage locations.
- Uploaded files are validated for size and permitted type.
- Database stores file metadata/path rather than unnecessary binary content.
- Generated laboratory PDFs are associated with lab_requests.
- Sensitive files are served through authorized application access rather than unrestricted public paths.

14. Logging, Audit & Observability

14.1 Audit Record

Field	Requirement
user_id	Authenticated actor
module	Target HMS module
request_ip	Source IP
timestamp	Action time
action	Operation performed
payload	Modified action payload subject to sensitive-data controls

14.2 Critical Audited Events

- Authentication events.
- Patient creation/update.
- Appointment changes.
- IPD admission and bed transfer.
- Prescription creation.
- Invoice/payment actions.
- Inventory adjustments.
- Commission processing.
- Withdrawal requests.
- Role/permission changes.
- Administrative configuration changes.

15. Backup, Recovery & Data Integrity

15.1 Backup Policy

- Cron executes daily at 00:00 UTC.
- MySQL backup is written to encrypted local/cloud storage.
- Backup failure is logged.
- Retention policy is configured operationally.
- Restore testing is required periodically.

15.2 Recovery Procedure

Recovery documentation must cover database restoration, application configuration restoration, file storage recovery, queue restart and post-restore integrity checks.

16. Vue.js Frontend Technical Requirements

16.1 Application Structure

- Vue Router for navigation.
- Pinia for user, permission and operational state.
- Axios service layer.
- Reusable forms, tables, dialogs, pagination and notifications.
- Permission-aware route guards.
- Responsive desktop/tablet layouts.
- Consistent loading, validation, empty and error states.

16.2 Functional Frontend Domains

Domain	Representative Screens
Authentication	Login, session, logout
Dashboard	Role-specific KPIs and operational summaries
Patients	Registration, search, profile, history
Appointments	Calendar, booking/registration, token queue
OPD/IPD	Consultation, admission, treatment, discharge
Beds	Occupancy, allocation, transfer
Billing	Invoice, receipts, payment history
Pharmacy	Inventory, barcode, billing
Laboratory	Tests, requests, samples, reports
Accounting	Income/expense, ledger, reports
Franchise	Referrals, wallet, withdrawals
Administration	Users, roles, permissions, configuration

17. API / Frontend / Mobile Interaction Contract

17.1 Request Lifecycle

- HTTPS request enters API.
- Sanctum authentication executes.
- RBAC/policy authorization executes.
- Request validation executes.
- Controller delegates to service/domain logic.
- Transaction executes when required.
- Jobs/events/notifications dispatch.
- Audit entry is written for critical actions.
- Structured response returns to client.

17.2 Error Handling

- Validation errors are field-specific.
- 401 for unauthenticated access.
- 403 for unauthorized access.
- 404 for missing resources.
- Conflict responses for invalid concurrent/state operations.
- Unexpected errors are logged server-side and returned as safe production messages.

18. Testing & Quality Assurance

18.1 Test Layers

Layer	Coverage
Unit	Services, calculations, commission rules, token helpers
Feature/API	Authentication, authorization and endpoint behavior
Integration	Transactions, invoices/ledger, bed transfer
Mobile	Login, API sync, FCM and doctor workflows
UI	Critical Vue workflows and permission navigation
Security	RBAC bypass, validation, unauthorized/file access
UAT	End-to-end hospital workflows

18.2 Critical Test Cases

- Concurrent appointment requests cannot duplicate daily tokens.
- Concurrent bed allocation cannot occupy the same bed twice.
- Bed transfer rolls back on failure.
- Invoice totals are server-calculated.
- Ledger is not written if billing transaction fails.
- Unauthorized users cannot access restricted modules.
- Doctor cannot access unauthorized doctor/patient resources.
- Referral commission cannot be credited twice.
- Withdrawal cannot exceed eligible wallet balance.
- Failed SMS/FCM jobs are retryable and observable.

19. Deployment & DevOps

Component	Requirement
OS	Ubuntu LTS
Web	Nginx reverse proxy
Runtime	PHP 8.x-fpm

Component	Requirement
Database	MySQL 8.0
Queue	Redis / Laravel worker
Scheduler	Laravel scheduler + cron
TLS	SSL/HTTPS
Source	GitHub
Storage	Controlled application/local/cloud storage

19.1 Deployment Sequence

- Provision server.
- Configure Nginx/PHP-FPM.
- Configure MySQL and Redis.
- Deploy Laravel from controlled Git branch.
- Configure protected environment secrets.
- Run migrations.
- Configure storage permissions.
- Configure queue workers and scheduler.
- Install SSL.
- Run smoke tests.
- Enable production traffic.

20. Performance & Scalability

- Paginate large collections.
- Index high-frequency search and relationship fields.
- Avoid N+1 database queries.
- Use queues for external operations.
- Keep API payloads focused.
- Keep transactions limited to critical units of work.
- Move expensive report/PDF operations out of synchronous requests where appropriate.
- Maintain API compatibility for future clients.

21. 25-Working-Day Implementation Plan

Days	Workstream	Deliverables
1-2	Foundation	Laravel/Vue setup, Git, environments, DB foundation, API conventions
3-4	Auth & RBAC	Sanctum, users, roles, permissions, middleware
5-6	Patients	Registration, UHID, history
7-8	Appointments	Lifecycle, daily token, reminders
9-10	OPD/IPD	Admission, treatment, bed management
11-12	Bed	Atomic transfer, occupancy
13-14	Billing	Invoices, items, GST, payments/ledger
15-16	Pharmacy	Inventory, barcode, stock/expiry
17	Laboratory	Tests, requests, PDF reports
18	Franchise	Referral, commission, wallet, withdrawal
19	Android	Doctor login, appointments, prescriptions, FCM
20	Notifications	Queue workers, SMS/FCM
21	Audit/Backup	Audit logs, backup, recovery procedure
22	Frontend	Dashboards, forms, tables, responsive integration

Days	Workstream	Deliverables
23	Testing	API, security, transaction and workflow testing
24	UAT	Staging deployment, UAT fixes, smoke tests
25	Production	Live deployment, verification, handover

22. Acceptance Criteria

- All protected APIs require Sanctum authentication.
- RBAC is enforced server-side.
- UHID follows UHID-YYYYMMDD-XXXX and is unique.
- Patient history aggregates prescriptions, lab tests and IPD admissions.
- Bed matrix correctly reports ICU, General, Private and Deluxe states.
- Bed transfer is atomic.
- Daily appointment tokens are concurrency-safe.
- Appointment reminders run asynchronously.
- Invoices are itemized and GST-aware with ledger coordination.
- Pharmacy supports pagination, barcode, low-stock and expiry warnings.
- Laboratory PDF reports are generated and associated with requests.
- Commission rules support Doctor-wise, Service-wise, Fixed and Surgery-wise models.
- Wallet and withdrawal operations are transactionally controlled.
- Doctor Android app consumes documented doctor APIs.
- FCM is operational.
- Daily encrypted backups are configured and restore procedure documented.
- Critical operations produce audit records.
- Production uses HTTPS and protected secrets.
- Critical workflows have automated tests.

23. Open Decisions & Explicit Assumptions

The supplied Master TRD is authoritative for the stated architecture and requirements. The following items must be finalized before production implementation rather than silently assumed:

- Exact Laravel/PHP versions at project start.
- Android language: Kotlin or Java.
- SMS gateway provider and templates.
- Cloud provider and production server sizing.
- Exact GST rules/rates and invoice numbering policy.
- Payment gateway and reconciliation process, if required.
- Commission approval and payout settlement method.
- Backup retention and geographic redundancy.
- Clinical-data retention/archival policy.
- Complete endpoint-by-endpoint role/permission matrix.
- Exact laboratory and invoice report templates.
- Whether multi-hospital/branch tenancy is required; it is not specified in the supplied TRD.
- Whether public appointment booking is required; the supplied TRD defines the endpoint but not the public booking UX.

24. Engineering Definition of Done

- Source code is committed to GitHub.
- Migrations and reference data are version-controlled.
- APIs have validation and authorization coverage.
- Critical transactions have automated tests.
- No production secrets are hard-coded.
- Queue workers and scheduled jobs are operational.

- Audit logging is enabled for critical operations.
- Daily backup is configured and tested.
- Staging/UAT is completed before production.
- Production smoke tests pass.
- API and operational documentation are handed over.